

# From Diagnosis to Cure: Decomposing the Tile-SPMD Abstraction Regret on Irregular Automata

Anonymous

**Abstract**—Tile-based GPU DSLs (Triton) trail hand-written CUDA by  $\sim 10\times$  on irregular finite-automata workloads. Prior work attributed this “abstraction regret” to the execution paradigm (tile/SPMD vs. thread-SIMT) rather than abstraction height. We turn that diagnosis into an anatomy and a cure: we decompose the regret into three independently measured components and identify, to the instruction level, the single missing primitive. On a work-efficient NFA worklist the  $10\times$  anchor decomposes as (A) a launch-configuration artifact ( $\sim 3.4\times$ , default `num_warps`), (B) a lane-packing-recoverable component (warp-redundant scalar execution), and (C) an irreducible  $\sim 2\times$  residual. Component C is *not* instruction count, occupancy, or memory bandwidth: at matched occupancy and *fewer* warp-instructions than CUDA, and sitting below both the issue and bandwidth roofline ceilings, a per-lane Triton worklist still spends  $15.3\times$  more cycles in the dependent-load stall and issues at 9.9% vs. 41% of peak—it is latency-bound. The root cause is abstraction-denied intra-warp memory-level parallelism: a CUDA warp’s lanes issue independent in-flight loads that overlap, whereas a lock-step tile serializes the dependent next-state load. We show the residual is *regime-dependent*: for the memory-bound DFA it closes (lane-packed Triton matches CUDA,  $1.05\times$ , once the table spills to DRAM). We specify the missing IR primitive—a `scalar_program` region that lowers each lane to an independent instruction stream—bound its payoff, and give the thread-model existence proof. Every kernel is correctness-gated against a CPU oracle; every number traces to a versioned CSV.

## I. INTRODUCTION AND CONTRIBUTIONS

A companion study [1] shows that on irregular automata the Triton→CUDA gap follows the execution paradigm, not the abstraction height. It leaves open *why*, mechanistically, and *what* would close it. This paper answers both. Our contributions:

- 1) A **falsifiable decomposition** of the tile-SPMD automata regret into artifact, recoverable, and irreducible, each measured and Nsight-attributed (Sec. III, Fig. 1).
- 2) Identification of the irreducible residual as **abstraction-denied intra-warp latency hiding**—not redundancy, occupancy, instruction count, or bandwidth—*measured*: fewer warp-instructions than CUDA, same occupancy, below both roofline ceilings, yet  $15.3\times$  the dependent-load stall and 9.9% vs. 41% issue activity (Sec. III-D).
- 3) A demonstration that the residual is **regime-dependent** (latency-bound NFA vs. memory-bound DFA), unifying both faces through one latency mechanism (Sec. IV).
- 4) A **launch-configuration finding** (default `num_warps=4` inflates the worklist regret  $\sim 3.4\times$ ),

a methodological caution for DSL benchmarking (Sec. III-B).

- 5) The **missing-primitive design** + a bound on its payoff + the thread-model existence proof, distinguished from Gluon’s per-thread *layout* (Sec. V).

## II. BACKGROUND AND METHOD

NFA simulation (a work-efficient worklist iterating the active set via `ffs` over a bitmask) is control-flow/latency-bound; DFA simulation (one dependent table gather `trans[cur*256+sym]` per symbol) is memory-bound. CUDA and NVIDIA Warp [2] are thread-SIMT (one thread per string); Triton [3] is tile/SPMD (a program operates on tiles). Correctness gates speed: every kernel/config is validated bit-for-bit against a CPU oracle before any throughput is reported. We report medians over warmup+9 reps at a GPU-saturating batch [4] (timings on GPUs are non-Gaussian, so we avoid  $\text{mean}\pm\text{std}$  and best-of- $N$ ), and sweep batch because the lane-packing benefit is occupancy-gated. Every mechanism claim is backed by Nsight Compute, not just wall-clock: the issue-stall composition (the `long_scoreboard` reason isolates dependent-load waits), achieved occupancy, issue activity, warp- and thread-instruction counts, and DRAM/L2 traffic. We report negative and partial results and correct two of our own intermediate hypotheses (Secs. III, III-D). Hardware: RTX 4070 (sm\_89, 46 SMs), Triton 3.5.1, torch 2.9.1+cu128, CUDA 13.3 / driver 580.

## III. THE DECOMPOSITION (NFA WORKLIST)

### A. The anchor

A work-efficient register-resident worklist ( $\leq 64$  states, batch 4096, 12 configs): Triton 22 Gbps vs. CUDA 227 Gbps =  $10.1\times$  **median** (9.4–12.3 $\times$ ).

### B. Component A: launch-configuration artifact

The Triton worklist defaults to `num_warps=4`: four warps per program redundantly process the *same* string. Sweeping `num_warps`  $\in \{1, 2, 4, 8\}$ : `nw=1` vs. `nw=4` =  $2.77\times@4096 \rightarrow 3.44\times@16384 \rightarrow 3.69\times@65536$  (each doubling  $\sim$ halves throughput). This inflates prior worklist regret numbers and must be disclosed.

### C. Component B: warp redundancy, recoverable by lane-packing

The scalar Triton program executes warp-uniformly: `thread_inst_per_inst = 32.00` (Triton) vs. 30.34 (CUDA), the same number with opposite meaning—CUDA’s 32 lanes run 32 *different* strings, Triton’s run the *same* one—yielding  $\sim 90\times$  more warp-instructions. Lane-packing (32 strings into the 32 lanes, exploiting the shared CSR so inner-loop bounds stay scalar) removes it. On the dense scan, pure lane-packing recovers  $3.2\times \rightarrow 9.8\times \rightarrow 19.4\times$  (batch 4096/16384/65536) toward the ideal  $32\times$ ; it is **occupancy-gated** (Fig. 2). *Correction*: we first framed the  $90\times$  warp-redundancy as the bottleneck; Nsight shows it is removed  $\sim 26\times$  yet throughput moves only  $3.8\times$ —it was largely hidden by occupancy.

### D. Component C: the irreducible residual

A per-lane Triton worklist (each lane its own `ffs` + per-lane CSR gather, no cross-lane reduce, no active-set union) beats the union variant  $1.27\times$  but is still  $0.51\times$  of CUDA ( $\approx 2\times$  slower). Nsight is decisive (Fig. 3): it has *fewer* warp-instructions than CUDA (4.66M vs. 5.07M) and the same occupancy (22.5%), yet is  $3.3\times$  slower because issue activity is 9.9%. Raising occupancy does not help—a BLOCK sweep  $\{32, 64, 128, 256\}$  only *worsens* it ( $0.49\times \rightarrow 0.31\times$ ): bigger lock-step tiles add divergence. **Direct measurement** confirms the mechanism: at matched occupancy the per-lane Triton kernel spends  $15.3\times$  more cycles in the `long_scoreboard` stall (waiting on a dependent memory load) than CUDA, and the issue-rate ratio ( $4.1\times$ ) tracks the throughput ratio ( $3.5\times$ ), consistent with Little’s law [5], [6]. The instruction roofline (Fig. 4) [7] settles “did you write a worse kernel?”: both kernels sit far below the issue *and* bandwidth ceilings (8.3%/27% vs. 32%/3.4% of peak), and Triton issues fewer warp-instructions yet runs  $3.5\times$  slower—it is structurally latency-bound. *Honest refinement*: the per-lane kernel does not issue *fewer* memory requests—it issues  $26\text{--}84\times$  *more* (masked inactive-lane gathers)—so the tile model pays twice: excess masked traffic *and* no dependent-load hiding. Root cause: a CUDA warp’s lanes are independent in-flight-load generators (memory-level parallelism); a lock-step tile serializes the dependent next-state load, so latency hides only across warps (occupancy-bound). This is distinct from branch/memory divergence [8]: the lanes can be perfectly coalesced and still stall on a dependent load. We distinguish it from the *hardware* fix of intra-warp stalls [9]: the same SM is fast under CUDA and slow under Triton at equal occupancy and fewer instructions, so the loss is abstraction-imposed.

Table I summarizes the multiplicative decomposition: removing the launch artifact and the warp redundancy each recovers  $3.7\times$ , leaving the irreducible  $1.9\times$  latency residual. By Little’s law (concurrency = throughput  $\times$  latency), the latency-bound kernel’s throughput is set by the in-flight-request concurrency it can sustain; the per-lane Triton kernel’s 9.9% issue activity and  $15.3\times$  dependent-load stall mean it sustains far less effective concurrency than CUDA at the

TABLE I  
THE WORKLIST REGRET DECOMPOSES MULTIPLICATIVELY (BATCH 16384,  $\leq 64$  STATES, ORACLE-GATED). EACH STAGE REMOVES ONE COMPONENT;  $28 \rightarrow 735$  IS  $3.7 \times 3.7 \times 1.9 = 26\times$ .

| Kernel                                | Gbps | Component removed                   |
|---------------------------------------|------|-------------------------------------|
| Triton worklist ( <code>nw=4</code> ) | 28   | —                                   |
| + <code>nw=1</code>                   | 104  | launch artifact ( $3.7\times$ )     |
| + lane-packing                        | 383  | warp redundancy ( $3.7\times$ )     |
| CUDA worklist (thread-SIMT)           | 735  | irreducible latency ( $1.9\times$ ) |

same occupancy, and the issue-rate ratio ( $4.1\times$ ) tracking the throughput ratio ( $3.5\times$ ) is exactly the relation Little’s law predicts when latency is fixed.

### IV. REGIME-DEPENDENCE (DFA): THE UNIFICATION

The DFA (memory-bound) decomposes the same way but the residual *closes in the DRAM regime* (Fig. 5). Across table sizes (cache $\rightarrow$ L2 $\rightarrow$ DRAM): scalar Triton DFA is flat  $\sim 29$  Gbps (independent confirmation of the scalar-gather ceiling); lane-packing gives  $\sim 12\times$  over it; and the lane-packed/CUDA ratio is  $0.55\text{--}0.62\times$  in cache but  $1.05\times$  at a 16 MB ( $>$ L2) table—lane-packed Triton *matches* CUDA. Mechanism (DRAM utilization only 14.6%, so memory-latency not saturated bandwidth): at moderate (cache) latency CUDA’s intra-warp hiding wins  $\sim 1.7\times$ ; at DRAM latency both must hide latency via cross-warp parallelism and *converge*. So the NFA’s fundamental  $\sim 2\times$  residual and the DFA’s closeable gap are one mechanism at different latency scales.

### V. THE CURE: THE MISSING PRIMITIVE

**Design.** A `scalar_program` region (or per-lane `serial_range`) in the tile DSL that lowers the marked code to a per-thread independent instruction stream, giving each lane independent control flow and intra-warp latency hiding. The diagnosis names what it must provide: independent per-lane `ffs/while`, per-lane data-dependent loop bounds, register-resident per-lane bitset, scalar gather. **Bound.** It closes the latency-bound residual (up to  $\sim 2\times$  on the NFA) and nothing extra in the already-converged DRAM-DFA regime—a regime-dependent, falsifiable prediction. **Existence proof.** The thread model realizes it: CUDA achieves  $1.0\times$  and the high-level Warp DSL [2] reaches  $0.9\times$ . **Not already in Gluon.** A runnable probe shows Triton’s low-level Gluon frontend exposes per-thread *layout* [10], not per-lane *control flow*: `gl.load` returns a layout-typed block, never a scalar, so the data-dependent CSR loop cannot even be lowered (compile error captured verbatim). Layout control  $\neq$  control-flow divergence.

#### A. Generality (positioning)

The same per-lane capability recurs across irregular GPU workloads (Table II); each needs per-lane data-dependent control flow with scatter/gather that lock-step tiles gate the same way. The contribution is a *capability* result; we keep the empirical depth on automata.

TABLE II  
THE SAME MISSING PER-LANE PRIMITIVE ACROSS IRREGULAR  
WORKLOADS (POSITIONING).

| Workload                    | Per-lane irregularity                               | Missing capability                             |
|-----------------------------|-----------------------------------------------------|------------------------------------------------|
| NFA/DFA automata            | active-set <code>ffs</code> ;<br>data-dep. CSR loop | per-lane <code>while</code> +<br>scalar gather |
| Var-length / entropy decode | data-dep. bit consumption                           | per-lane loop +<br>bit-cursor                  |
| Ragged batch / MoE routing  | variable seq. length;<br>routing                    | per-lane bounds +<br>scatter                   |
| Sparse frontier / BFS [11]  | variable vertex degree                              | per-lane neighbor<br>loop + scatter            |
| Tokenization / lexing       | data-dep. longest-match                             | per-lane DFA scan                              |

## VI. METHODOLOGICAL INTEGRITY

Our analysis corrected two of its own intermediate hypotheses, which we report rather than hide. First, an early reading framed the  $\sim 90\times$  warp-instruction redundancy (Sec. III) as the bottleneck; profiling the lane-packed kernel showed the redundancy is removed  $\sim 26\times$  yet throughput moves only  $3.8\times$ —it was largely hidden by occupancy, so warp-redundancy is real but not load-bearing. Second, we predicted (following the latency-hiding literature) that the per-lane Triton kernel would sustain *fewer* in-flight memory requests than CUDA; direct measurement refuted this—it issues  $26\text{--}84\times$  *more* (masked inactive-lane gathers) and still stalls, so the residual is the inability to *hide* dependent-load latency, not a request-count deficit. We therefore phrase the mechanism via stall composition and issue activity, not request counts. Both corrections sharpened, rather than weakened, the central claim.

## VII. THREATS TO VALIDITY

*Single GPU.* Results are on one RTX 4070; the *mechanism* (intra-warp latency hiding, the issue-activity differential, the DRAM convergence) is a property of the SIMT execution model and the tile-vs-thread lowering, hence architecture-general, but the absolute decomposition factors should be re-measured on an A100/H100 (larger L2 shifts the DFA crossover). *Prototype scope.* A multi-word generalization of the lane-packed kernel (NWORDS int64 words/lane) is oracle-validated to 256 states, so the prototype is not artificially limited; but the clean  $\sim 2\times$  residual is a *register-resident-regime* ( $\leq 64$ -state) result—past 64 states the CUDA register worklist itself degrades (register pressure), so the head-to-head is confounded by both baselines losing the register-resident advantage, and ANMLZoo-scale automata (Brill 42661 = 667 words) make the per-lane multi-word scatter explode—itsself a manifestation of the tile limitation we study. Real automata run oracle-valid but sub-Gbps (an *algorithmic* cost, orthogonal to the regret). *Tuning.* `num_warps` is a knob; we disclose the full sweep rather than a single config, rebutting the autotuning counter-thesis [12]: the Triton/Gluon same-MLIR control plus the disclosed sweep separate *tuning* (which closes component A and part of B) from *expressibility* (the residual C), with a probe that is falsifiable by construction.

## VIII. REPRODUCIBILITY

Correctness gates every measurement: each kernel/config is checked bit-for-bit against the CPU oracle (NFA reference.py / simulate\_dfa) before any throughput is reported. Every number in this paper traces to a versioned CSV; all figures regenerate from those CSVs by a single script, so the plots cannot drift from the measurements. The Gluon expressibility claim is a runnable probe that exits 0 on the expected compile failure and 1 if a future Gluon ever compiles it—falsifiable by construction. The artifact (kernels, oracle, sweeps, profiling commands, CSVs) supports one-command regeneration.

## IX. RELATED WORK

**GPU automata** form an *algorithmic* axis, all single-implementation CUDA: ngAP’s non-blocking multi-symbol processing [13], HybridSA’s bit-parallel shift-and [14], Bit-Gen’s Parabix bitstreams [15], AsyncAP’s input-symbol asynchrony [16], AutomataBLAS’s AP-as-SpMV [17], and the Hopps sparsity ASIC [18]. Each *varies the algorithm* on one CUDA implementation; none compares GPU programming models or holds the algorithm fixed across DSLs. The sole IR-for-automata work [19] lowers regex to one fixed domain-specific accelerator—a hardware target, not a GPU-paradigm/expressibility study, and with no tile-vs-thread cost. Our axis is orthogonal: fixed algorithm, varied programming model. **Tile GPU DSLs:** Triton [3] and the 2024–26 wave exposing more thread/warp control. The closest, Gluon’s Linear Layouts [10], give per-thread *data placement* but not per-lane *control flow* (our probe shows the data-dependent loop cannot lower); Descend [20] makes the thread hierarchy first-class but offers no tile drop-to-scalar escape. Warp [2] is the thread-SIMT existence proof. **Mechanism:** we ground the residual in latency-hiding / MLP-vs-occupancy analysis [5], [6] and place both kernels on the instruction roofline [7]; we distinguish it from divergence [8] and from the *hardware* intra-warp-stall fix of Subwarp Interleaving [9] (we attribute the same stall to the DSL, at equal occupancy and fewer instructions). **Methodology and lineage:** rigorous benchmarking [4], roofline [21], and the performance-portability metric [22] (per-hardware; we measure per-*capability* regret); we rebut the autotuning counter-thesis [12] via the Triton/Gluon same-MLIR control and the disclosed `num_warps` sweep. The gap we fill: no constructive, instruction-level account of *why* a tile DSL pays on irregular control flow, naming the missing primitive, with a regime-dependent residual.

## X. CONCLUSION

The abstraction regret on irregular automata is not a monolith: a launch artifact, a recoverable redundancy, and an irreducible intra-warp-latency-hiding residual whose size is set by the latency regime. The cure is a named, bounded, existence-proven IR primitive; building it in Triton-MLIR is the landmark future step.

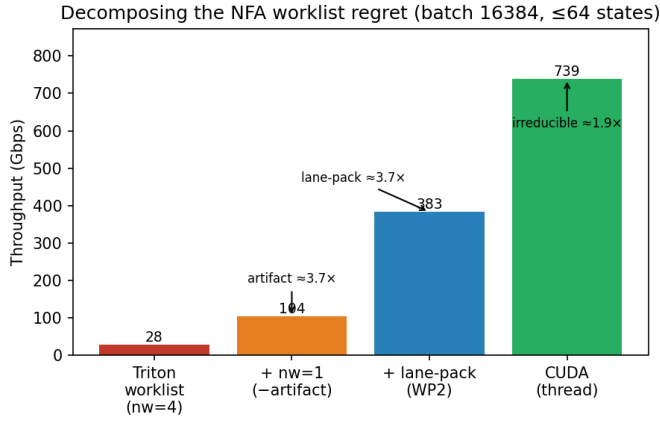


Fig. 1. NFA worklist throughput (batch 16384,  $\leq 64$  states) climbs as each regret component is removed:  $nw=4 \rightarrow nw=1$  (launch artifact),  $\rightarrow$  lane-packing (warp redundancy),  $\rightarrow$  CUDA (irreducible latency). All oracle-gated.

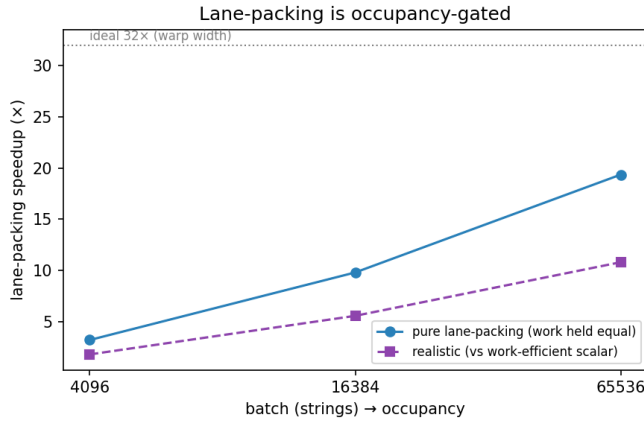


Fig. 2. Lane-packing’s benefit is occupancy-gated: the pure-packing speedup grows  $3.2\times \rightarrow 19.4\times$  with batch (toward the ideal  $32\times$ ) as more warps fill the GPU.

## REFERENCES

- [1] Anonymous. The two faces of abstraction regret: Execution paradigm over abstraction height on irregular automata, 2026. Companion diagnosis paper.
- [2] NVIDIA. Warp: A python framework for high-performance GPU simulation and graphics, 2022. <https://github.com/NVIDIA/warp>.
- [3] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL*, 2019.
- [4] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems. In *SC*, 2015.
- [5] Vasily Volkov. *Understanding Latency Hiding on GPUs*. PhD thesis, University of California, Berkeley, 2016. Tech. Rep. UCB/EECS-2016-143.
- [6] Sunpyo Hong and Hyesoon Kim. An analytical model for a GPU architecture with memory-level and thread-level parallelism awareness. In *ISCA*, 2009.
- [7] Nan Ding and Samuel Williams. An instruction roofline model for GPUs. *Concurrency and Computation: Practice and Experience*, 2022.
- [8] Wilson W. L. Fung, Ivan Sham, George Yuan, and Tor M. Aamodt. Dynamic warp formation and scheduling for efficient GPU control flow. In *MICRO*, 2007.
- [9] Sana Damani et al. GPU subwarp interleaving. In *HPCA*, 2022.

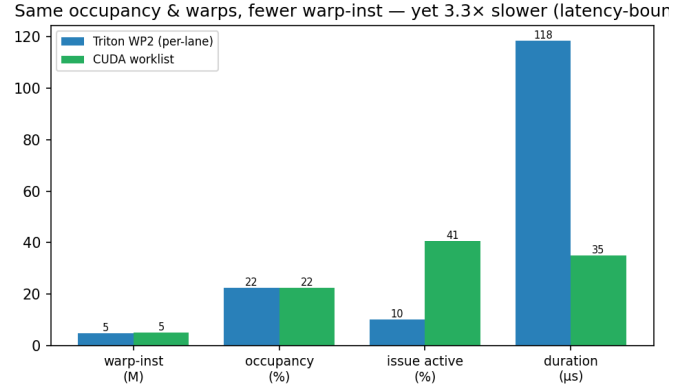


Fig. 3. Per-lane Triton vs. CUDA (16384 strings, 32 states): same occupancy, fewer warp-instructions, yet  $3.3\times$  slower at 9.9% vs. 41% issue activity—latency-bound.

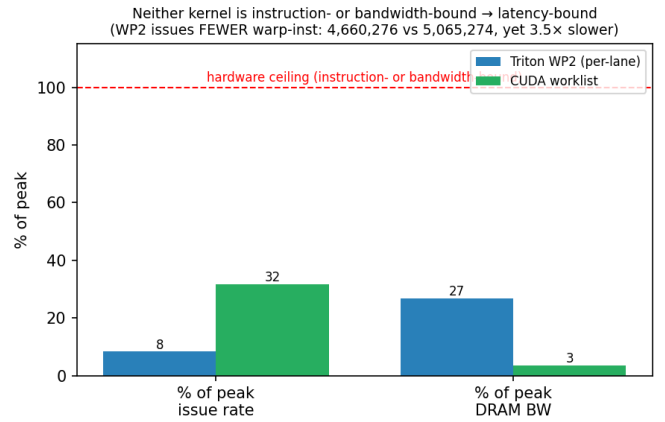


Fig. 4. Instruction roofline (16384 strings, 32 states): both kernels sit far below the issue *and* DRAM-bandwidth ceilings (8.3%/27% for Triton, 32%/3.4% for CUDA), so neither is instruction- nor bandwidth-bound—the residual is latency.

- [10] Triton contributors. Linear layouts: Robust code generation of efficient tensor computation using  $\mathbb{F}_2$ , 2025. arXiv:2505.23819.
- [11] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D. Owens. Gunrock: A high-performance graph processing library on the GPU. In *PPoPP*, 2016.
- [12] Burkhard Ringlein, Thomas Parnell, and Radu Stoica. GPU performance portability needs autotuning, 2025. arXiv:2505.03780.
- [13] Tianao Ge, Tong Zhang, and Hongyuan Liu. ngap: Non-blocking large-scale automata processing on GPUs. In *ASPLOS*, 2024.
- [14] HybridSA authors. HybridSA: GPU acceleration of multi-pattern regex matching using bit parallelism. *Proc. ACM Program. Lang. (OOPSLA)*, 2024.
- [15] Tianao Ge, Hongyu Chu, and Hongyuan Liu. Interleaved bitstream execution for multi-pattern regex matching on GPUs. In *MICRO*, 2025.
- [16] Hongyuan Liu, Sreepathi Pai, and Adwait Jog. Asynchronous automata processing on GPUs. *Proc. ACM Meas. Anal. Comput. Syst. (POMACS)*, 2023.
- [17] AutomataBLAS authors. Advancing matrix operations for high-performance and memory-efficient automata processing on GPUs. *ACM Trans. Archit. Code Optim. (TACO)*, 2025.
- [18] Yufeng Du, Joel S. Emer, and Daniel Sánchez. Hopps: Leveraging sparsity to accelerate automata processing. In *ASPLOS*, 2025.
- [19] Filippo Somaini, Luca Carloni, Giovanni Agosta, Marco D. Santambrogio, and Davide Conficconi. Combining MLIR dialects with domain-

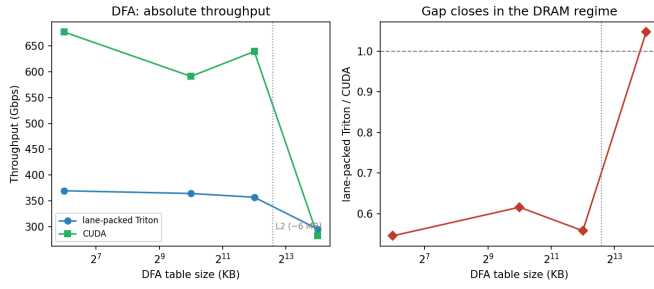


Fig. 5. DFA: lane-packed Triton trails CUDA  $\sim 1.7\times$  while the table is cache-resident but *matches* it ( $1.05\times$ ) once the 16 MB table spills past L2—both then rely on cross-warp parallelism. The residual is regime-dependent.

specific architecture for efficient regular expression matching. In *CGO*, 2025.

- [20] Bastian Köpcke, Sergei Gorlatch, and Michel Steuwer. Descend: A safe GPU systems programming language. In *PLDI*, 2024.
- [21] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Commun. ACM*, 2009.
- [22] S. J. Pennycook, J. D. Sewall, and V. W. Lee. A metric for performance portability, 2016. arXiv:1611.07409.